From Co-Pilot to Co-Worker

Marc Müller

25 August 2026

4tecture©
empower your software solutions



About me:

Marc Müller
Principal Consultant
@muellermarc

MVP
Microsoft Most Valuable Professional

4tecture©
empower your software solutions

Our Products:

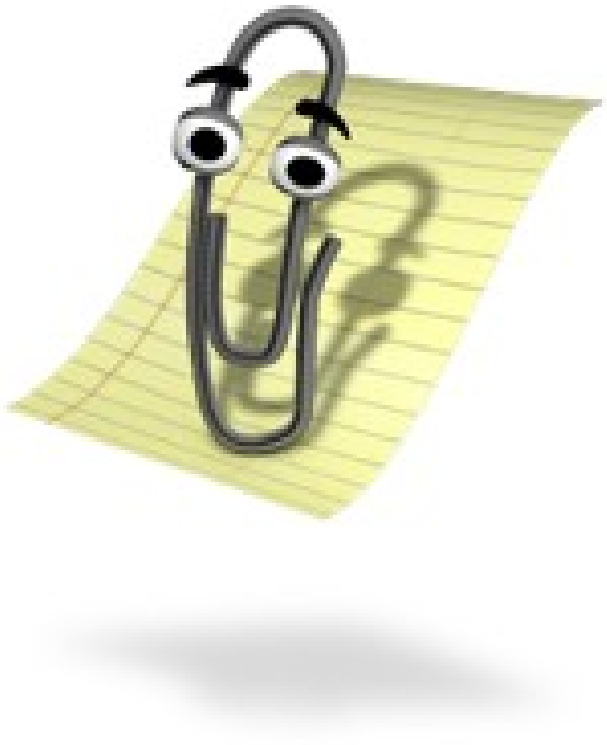
Multi-Tenant OpenID Connect Identity Provider

ProAuth
www.proauth.net

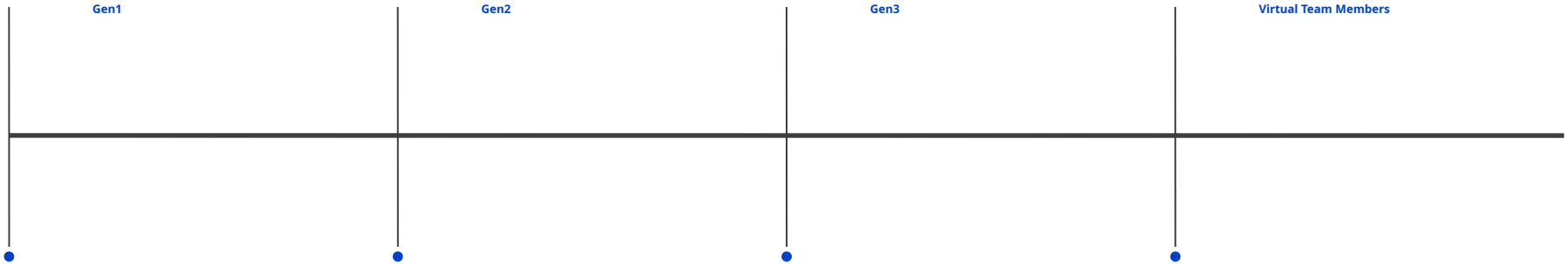
Enterprise Application Framework for .NET

ReaFx
www.reafx.net

Microsoft and its AI History

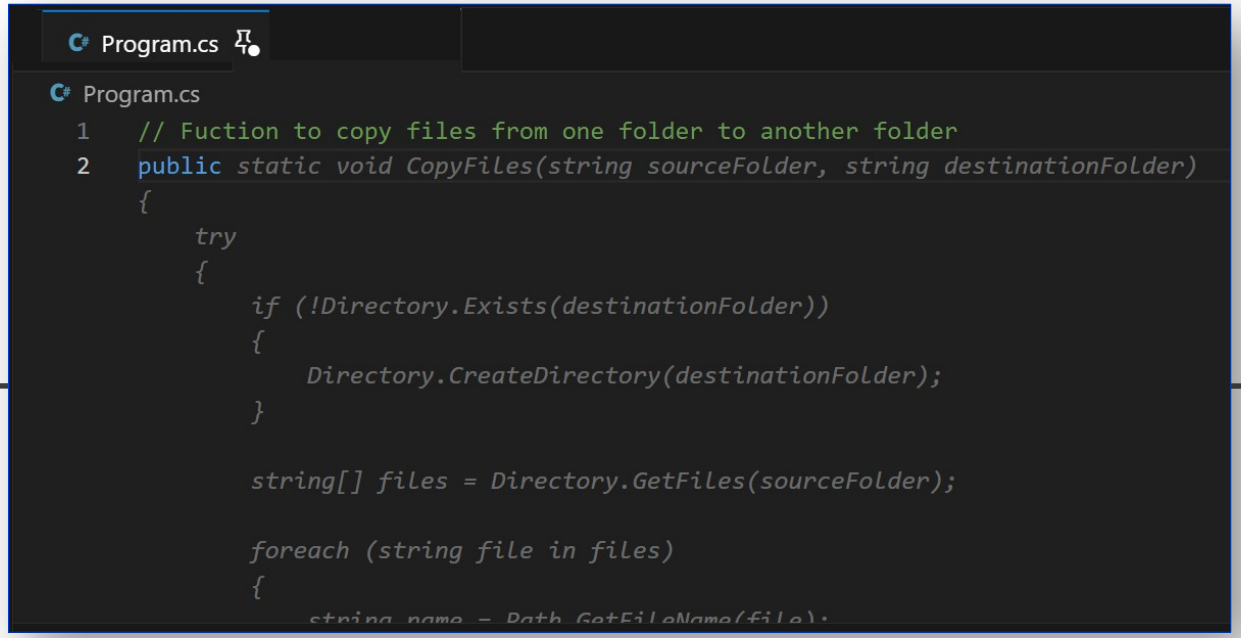


Timeline



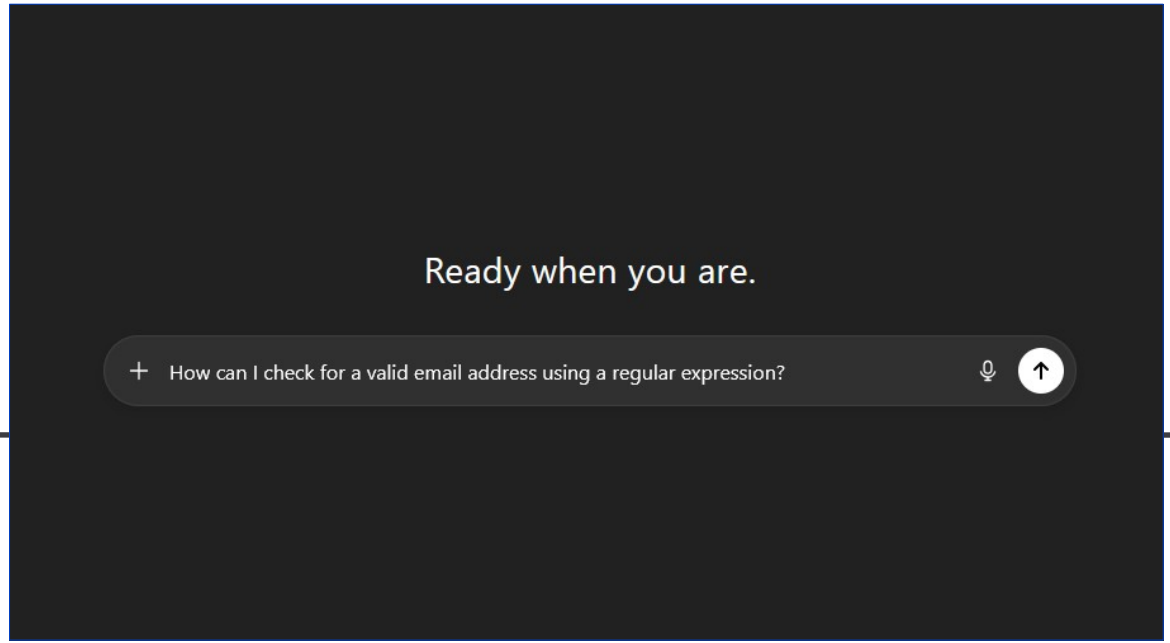
Timeline

Gen1



Timeline

Gen2



Timelin

Gen2

How can I check for a valid email address using a regular expression?

Thought for 12s ›

Short answer: don't try to perfectly match RFC-5322 with a regex. Use a *practical* regex to catch obvious typos, and rely on a confirmation email for the real validation.

Here are good options depending on how strict you want to be:

Quick & simple (good enough for many apps)

Catches missing `@`, spaces, and missing dot in the domain.

Regex

kotlin — Copy code

```
^[^\s@]+@[^\s@]+\.[^\s@]+$
```

JavaScript

js — Copy code

```
/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)
```

Python

py — Copy code

```
import re
```

+ Ask anything

Gen2

OK, so what do you suggest since you recommend to not perfectly match the RFC?

Great question. Here's a *practical* approach that catches real mistakes without blocking valid edge-case emails:

1) Client-side: light checks for UX

- **Use HTML5** and a **simple regex** to catch obvious typos.
- **Trim** input, lowercase the domain, and keep the local part as-is.
- **Don't** try to enforce perfect syntax on the client.

HTML

```html
<input type="email" name="email" autocomplete="email" required />
```

Light regex (good enough)

```js
const basicEmail = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
basicEmail.test(email);
```

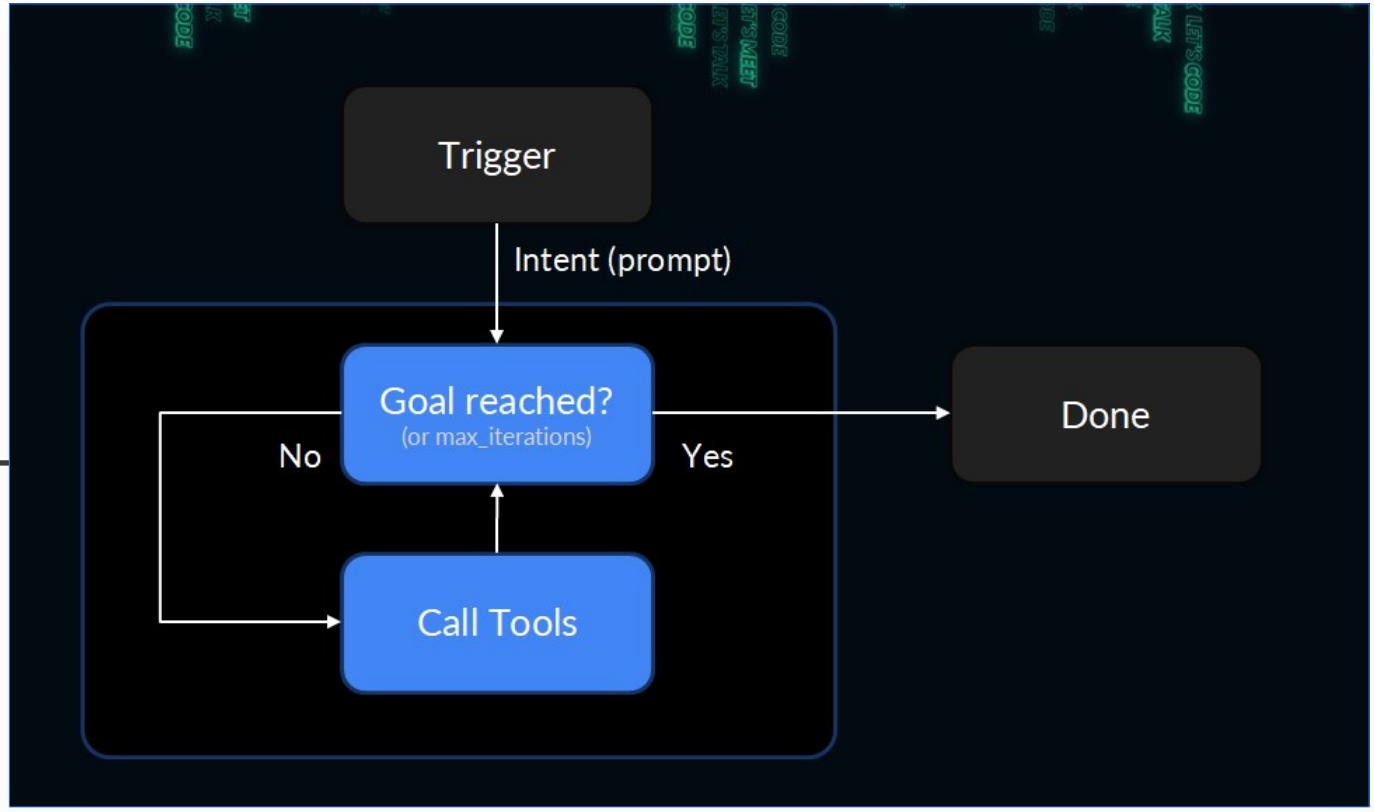
2) Server-side: sensible strictness

- Re-validate server-side with a stricter—but still sane—pattern.
- Enforce length limits (≤254 total, ≤64 before `@`).
- Allow common characters in the local part.
- Require reasonable domain labels and TLD.

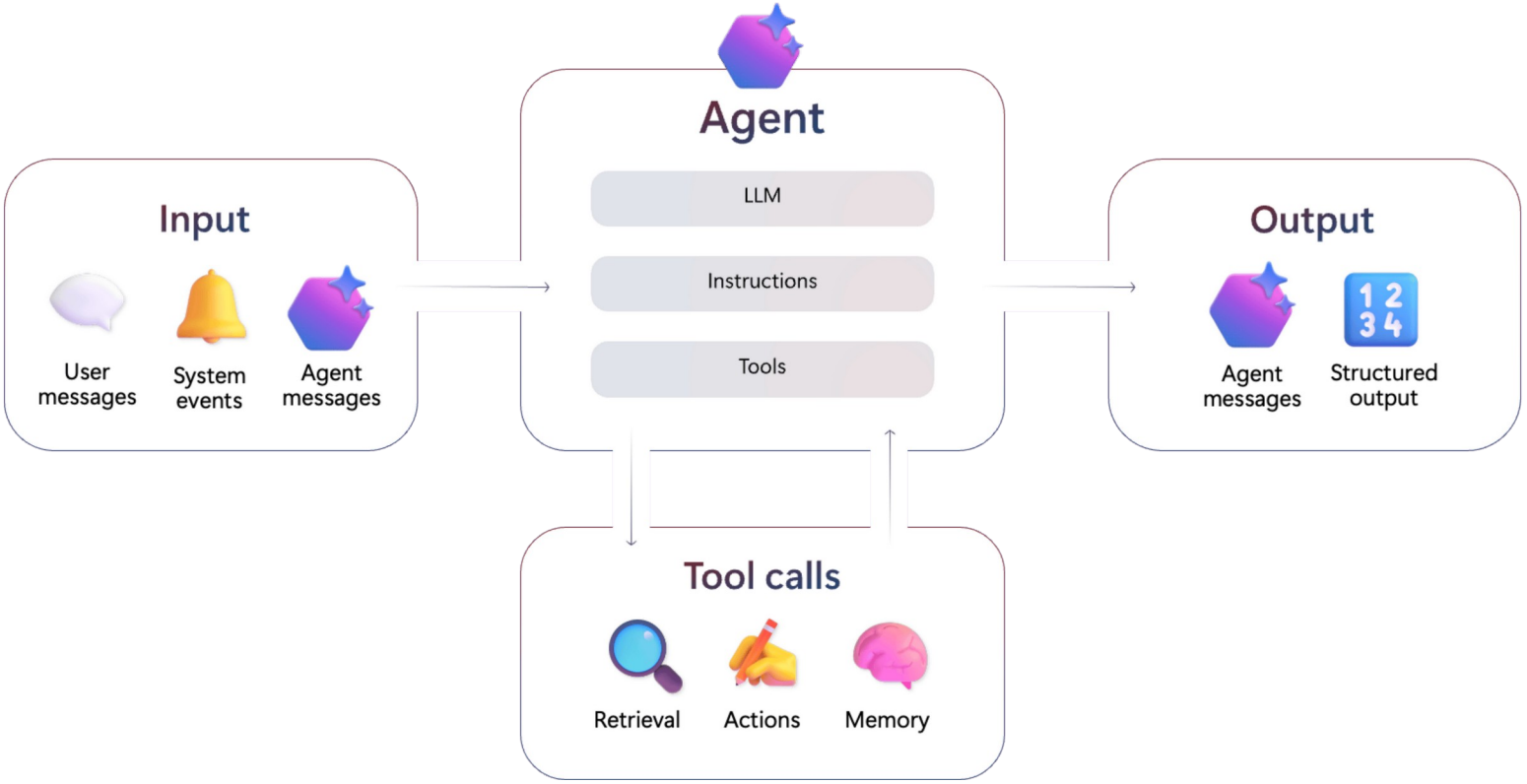
JavaScript/TypeScript (Node)

Timeline

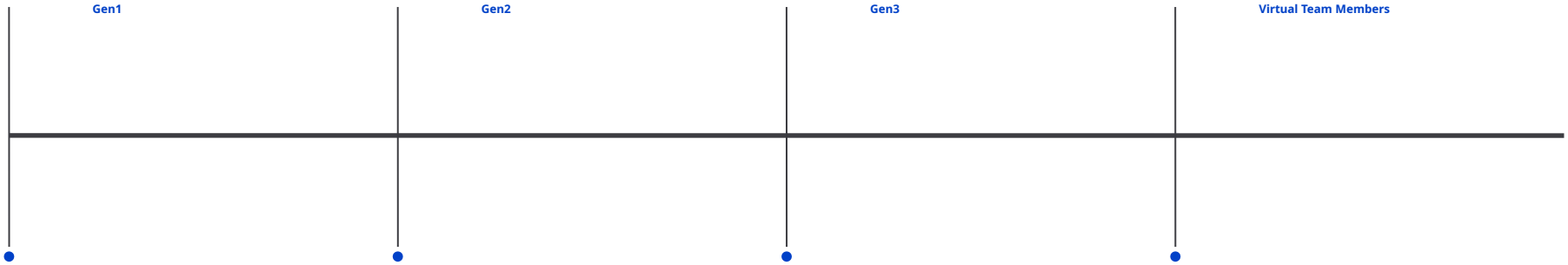
Gen3



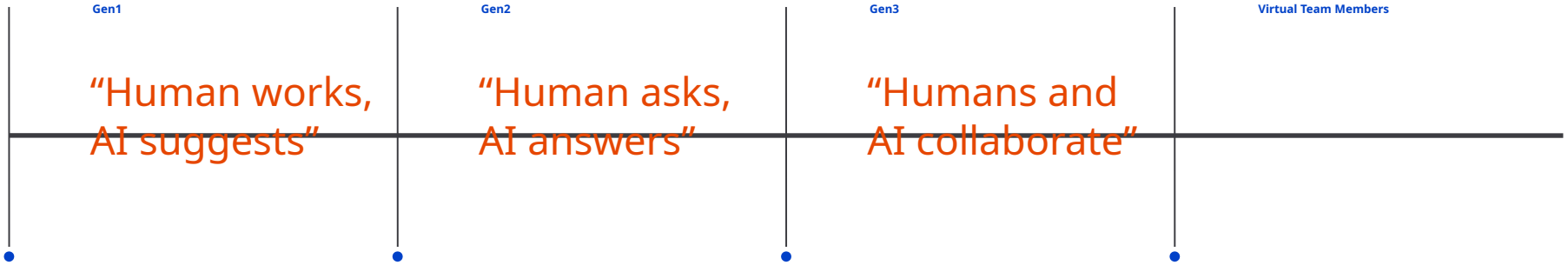
Agents (Marketing version)



Timeline



Timeline


Gen1
“Human works, AI suggests”
Gen2
“Human asks, AI answers”
Gen3
“Humans and AI collaborate”
Virtual Team Members

WE CHANGED THE UNIT OF WORK

The evolution of our job

- We learned to prompt. Now we must learn to delegate.
- A longer prompt does not create a better co-worker.

A prompt is not onboarding

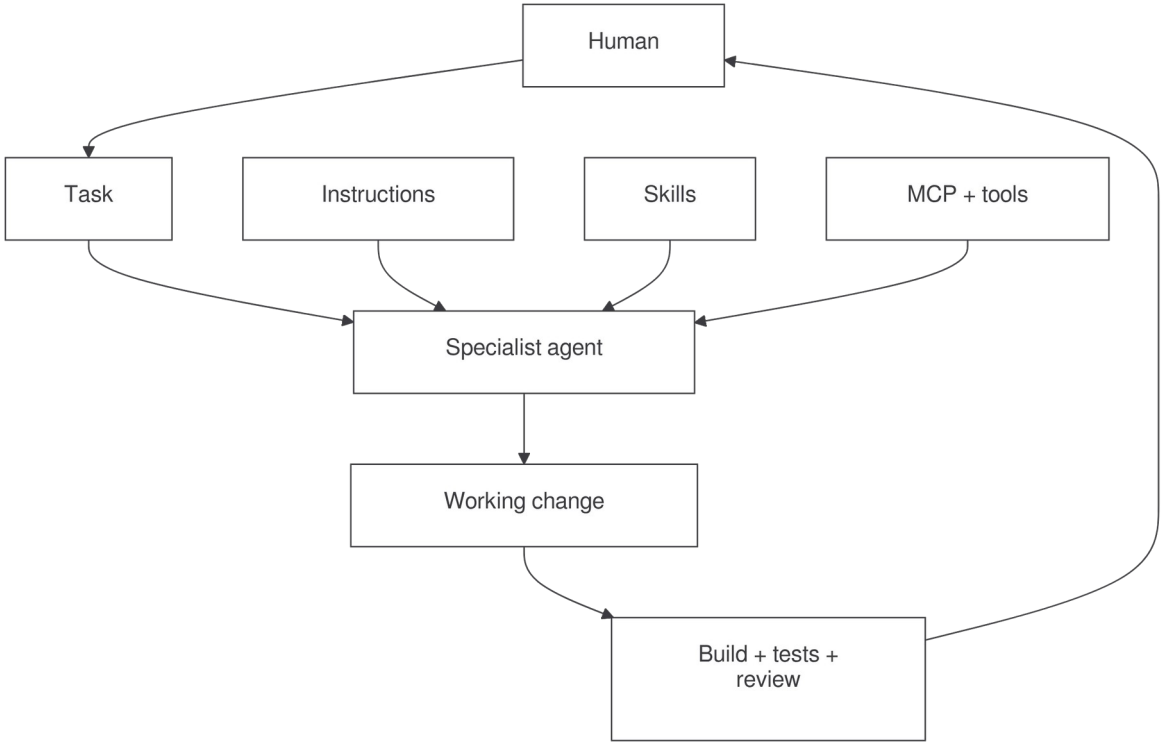
Prompt-only

- Repeats team knowledge
- Mixes goal and procedure
- Grows with every failure

Co-worker setup

- Documents stable knowledge
- Teaches reusable workflows
- Grants only needed tools

Build the worker, not the prompt



Put knowledge where it belongs

Always relevant

- Repository instructions
- Scoped language conventions

Loaded when needed

- Skills: reusable workflows
- Agents: roles and boundaries
- MCP: external capabilities

Access is part of the job description

Give every role the smallest useful toolbox.

- Investigator: browser + work items, no source edits
- Developer: code + terminal
- Reviewer: read-only checks, no fixes

DEMO 1 · DELEGATE THE INVESTIGATION

Customer feedback gives us intent

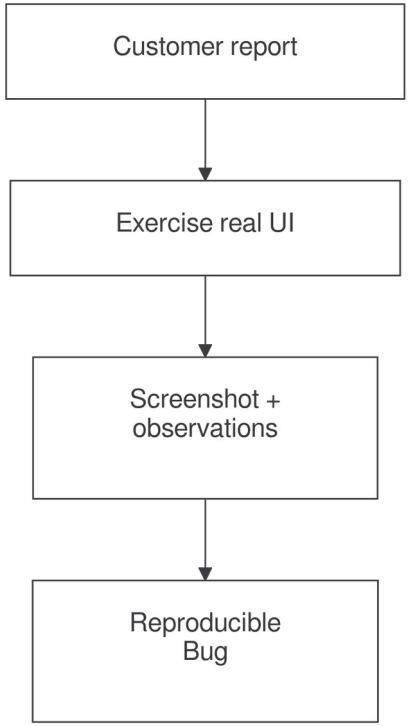
Feedback

“Quantities sometimes behave incorrectly when the same product is added repeatedly.”

Investigator contract

- Investigate
- Prove with evidence
- Create a professional Bug
- Do not modify the app

Browser evidence closes the reality gap



A Bug is a task contract

Evidence

- Repeatable steps
- Expected versus actual
- Screenshot in context

Decision support

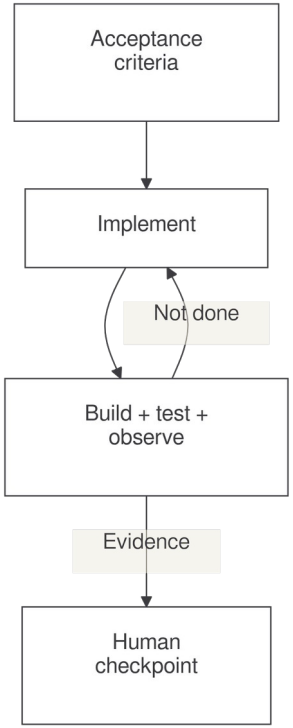
- Impact
- Labeled hypotheses
- Acceptance criteria

DEMO 2 · DELEGATE THE REPAIR

The issue tells us what. The repository teaches how.

- Add a failing regression first
- Audit every active order-processing implementation
- Make the smallest compatible fix
- Recreate the complete Compose topology

An agent needs feedback—not faith



DEMO 3 · MAKE THE BEHAVIOR DURABLE

Train once. Reuse on every regression.

The skill carries

- xUnit v3 conventions
- Page Object boundaries
- Observable waits
- Stable selectors

The task carries

- User journey
- Expected quantities
- Expected total

DEMO 4 · ASK FOR PEER REVIEW

Review the change — not just the last diff

Committed + staged + unstaged + untracked

- Trace every acceptance criterion
- Check architecture and compatibility
- Report actionable findings with evidence
- Return a qualified verdict

Automated loops need a qualified exit

Pass 1

Ready → Ready for PR

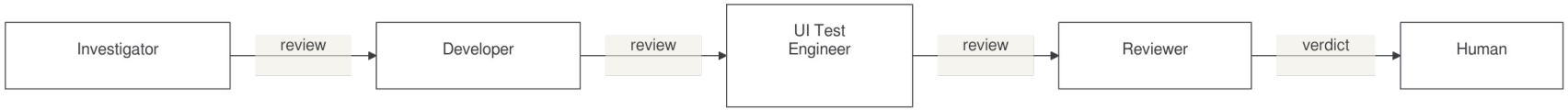
Changes required → One correction

Pass 2

Ready → Ready for PR

Findings remain → Human decision

Every handoff is a human checkpoint



Orchestration

- One workflow. Four specialists. One accountable human.
- The orchestrator can delegate the chain — without removing the gates.

OUTLOOK: SUPERVISION SCALES BEYOND ONE CHAT

Autonomy is a dial — not a switch

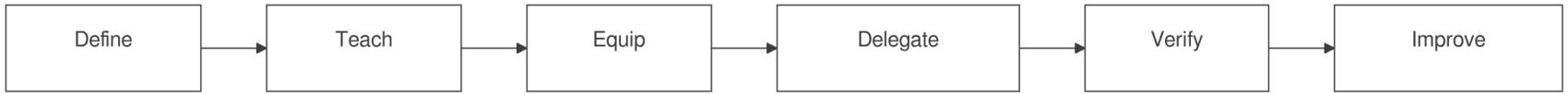
Interactive

- Local session
- Reviewed handoffs
- Fast feedback

Asynchronous

- Parallel sessions
- Background or cloud execution
- Pull request for review

The developer’s job moved up a level



A co-worker needs five things

1. **Knowledge** — compact, scoped instructions
2. **Practice** — reusable skills
3. **Access** — narrow tools and MCP
4. **Feedback** — builds, tests, runtime evidence
5. **Accountability** — review and qualified exits

Make the agent earn your trust

- Delegate the work.
- Keep the engineering bar
- **Stop typing. Start supervising.**

Thank you! Let’s continue the conversation

If you would like to discuss your challenges or explore how we can support you, please contact us.

4tecture GmbH
Industriestrasse 25
CH-8604 Volketswil
www.4tecture.ch

Marc Müller
Principal Consultant

4tecture©
empower your software solutions

ProAuth

ReaFx


4tecture©
empower your software solutions

References

- VS Code: Agent customization
- VS Code: Custom agents and handoffs
- VS Code: Agent Skills
- VS Code: Tools
- VS Code: Browser tools
- VS Code: MCP servers
- VS Code: Agent sessions
- GitHub: Copilot customization cheat sheet